

DailyMealBox — Security Architecture v1

Deliverable for: Amendment v2 Extra, Gap N (GDPR Art. 32), "Security_Architecture_v1.pdf"

Date: 2 October 2026

Status: Draft for DailyMealBox review. Production stays off until DailyMealBox approves this document and the pre-launch checklist (§8) is signed off.

Platform note. The SOP describes native Android apps. DailyMealBox is built as React web apps (customer, vendor, driver, admin, office), wrapped for Android, talking to one Node.js/Express API backed by MongoDB. So "the apps" in this document means browser or WebView clients on a single HTTPS origin. Media files are stored in Cloudinary, not Firebase Storage or Google Drive.

1. Summary

Requirement (Gap N)	Status	Where it is enforced
All traffic over HTTPS, HTTP redirected	✅ Built	API middleware (FORCE_HTTPS), HSTS header
TLS 1.3 minimum	⚙️ Hosting config	Load balancer / reverse proxy (§2.1)
WebSockets over WSS	✅ By design	Socket.IO shares the HTTPS origin
Database encrypted at rest (AES-256)	⚙️ Hosting config	MongoDB Atlas (§3)
PII field-level encryption (AES-256)	✅ Built for addresses, bank details, ID numbers, WhatsApp numbers, integration secrets · ⚠️ phone and email still plaintext (§4.4)	Application layer (§4)
Key management	✅ Built	Environment secret + rotation procedure (§5)
Driver documents restricted, 1-hour access	❌ Not yet (§6.2)	—
Access controls	✅ Built	§7
Live verification	✅ Built	Admin → Platform Controls → Security & encryption

2. Data in transit

2.1 TLS

- TLS is terminated at the load balancer / reverse proxy in front of the API and the web apps. **It must allow TLS 1.3 only** (on nginx: `ssl_protocols TLSv1.3;`). The application cannot enforce the protocol version itself. This is a deployment setting that must be confirmed before launch (§8).
- The proxy must pass `X-Forwarded-Proto` so the API can tell HTTPS from HTTP requests.
- Google Maps, payment gateways (Przelewy24, Stripe, Razorpay), Cloudinary, Mailchimp and WhatsApp providers are called over HTTPS only.

2.2 HTTP → HTTPS enforcement (`Backend/src/app.js`)

- `FORCE_HTTPS` is **on by default in production** (it can be set explicitly to `true/false`).
- `GET/HEAD` over HTTP get a **308 redirect** to the same URL on HTTPS.
- Every other method over HTTP is **refused with 403**. A login or payment body is never accepted over plain HTTP, so it is not sent twice.
- Health-check endpoints stay reachable over HTTP for the load balancer.

2.3 Security headers (helmet)

- **HSTS** in production: `max-age=31536000; includeSubDomains; preload`.
- `Content-Security-Policy: default-src 'self'` on API responses, `X-Content-Type-Options: nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`.

2.4 WebSockets

Socket.IO connects to the same origin as the page. Served over HTTPS, the connection is `wss://`. Browsers and WebViews block a `ws://` connection from an HTTPS page, so plain WS cannot happen in production.

2.5 Android wrapper

The wrapper should load only the HTTPS origin and disable cleartext traffic (`android:usesCleartextTraffic="false"` or a network security config with `cleartextTrafficPermitted="false"`).

3. Data at rest (database and backups)

- **MongoDB Atlas** holds all application data. Atlas encrypts cluster storage and backups at rest with AES-256. For a key DailyMealBox controls, enable **Encryption at Rest using Customer Key Management** with AWS KMS on the Atlas project. The spec allows either AWS KMS or MongoDB-managed keys; **choose one and confirm in writing before launch** (§8).
- The connection to Atlas must use TLS (`mongodb+srv://` , TLS by default). The admin Security page shows whether the configured connection string uses TLS and points at Atlas.

- Atlas network access: allow only the API servers' IPs; use a database user with least privilege (read/write on the application database only).

4. Field-level encryption (application layer)

4.1 Algorithm and format

- **AES-256-GCM**: 256-bit key, a fresh random 96-bit IV for every value, 128-bit authentication tag. Tampering or a wrong key is detected, not silently mis-decrypted.
- Stored form: `enc:v1:<keyId>:<iv>:<tag>:< ciphertext>` (base64). `keyId` is a fingerprint of the key that wrote the value, never the key itself.
- Implementation: `Backend/src/utils/fieldCrypto.js` (crypto) and `Backend/src/utils/encryptedFields.plugin.js` (Mongoose integration).

4.2 How it is applied

- A Mongoose setter encrypts on every write path: create, assignment + save, and `$set` in update queries.
- Reads decrypt on every read path: hydrated documents, `toJSON/toObject`, `.lean()`, `populate`, `findOneAndUpdate` and `aggregate` results. Application code and API responses see plaintext; **the database only ever holds ciphertext**, so someone with database access (a DBA, a leaked backup, Atlas support) sees `enc:v1:...`.
- A **start-up migration** runs on every server start. It encrypts any plaintext left in these fields (data from before this release, or written by tools that bypass the application), and re-encrypts values written with an older key after a rotation (§5.2). It writes only when the value has not changed meanwhile, so several API instances can start at the same time.

4.3 Encrypted fields

Data	Collection · fields
Customer addresses	<code>food_users</code> · <code>addresses[].street</code> , <code>addresses[].additionalDetails</code> , <code>addresses[].zipCode</code> , <code>addresses[].phone</code>
Customer WhatsApp number (invoice delivery)	<code>food_users</code> · <code>whatsappNumber</code>
WhatsApp invoice log recipient	<code>dmb_whatsapp_logs</code> · <code>to</code>
Driver bank details	<code>food_delivery_partners</code> · <code>bankAccountNumber</code> , <code>bankIban</code>
Driver identity-document numbers	<code>food_delivery_partners</code> · <code>drivingLicenseNumber</code> , <code>aadharNumber</code> (national ID), <code>panNumber</code> (tax ID)
Vendor bank account	<code>food_restaurants</code> · <code>accountNumber</code>
Fleet partner bank details	<code>fleet_partners</code> · <code>bankIban</code>

Data	Collection · fields
Kitchen partner bank details	kitchenpartners · bankDetails
Integration secrets entered in the admin panel	Mailchimp API key, Twilio auth token, 360dialog API key (stored only as ciphertext; secrets set as environment variables never reach the database)

The address city and map coordinates stay in plaintext: zone matching and route planning need them, and on their own they don't identify a person.

Payment cards: no card data, including last-4 digits, is stored by DailyMealBox. Payments run on the gateways' hosted pages (Przelewy24 / Stripe / Razorpay), and only the gateway's payment reference is kept. The spec's `user.payment_methods[]` does not exist in this system.

4.4 Deviation — customer phone and email (needs DailyMealBox decision)

The spec lists `user.phone` and `user.email` for field-level encryption. They are **not encrypted in this release**:

- The phone number is the **login identifier**. OTP login looks a user up by exact phone match under a unique index, and email is also used for lookups.
- Admin customer search matches partial phone numbers and emails. Encrypted values cannot be searched by prefix.
- Phone and email are also copied onto orders, invoices and notification logs. Those copies would all need the same treatment.

Proposed approach (Phase 2, ~3-4 days with testing): store each value encrypted plus a **blind index**, a keyed HMAC (`blindIndex()` is already in `fieldCrypto.js`). Login and duplicate checks use the blind index under a unique index. Admin search becomes exact-match only (full phone or full email). Copies on orders and invoices are encrypted the same way. Until then, phone and email are protected by Atlas encryption at rest, TLS, and database access restricted to the API servers.

5. Key management

5.1 Keys

Secret	Purpose	Format
PII_ENCRYPTION_KEY	Field-level encryption (§4)	32 random bytes, base64 or 64 hex chars — generate with <code>openssl rand -base64 32</code>
PII_ENCRYPTION_KEY_PREVIOUS	Only during a rotation (§5.2)	same
INVOICE_LINK_SECRET	HMAC signing of invoice PDF links	long random string

Secret	Purpose	Format
	(§6.1)	
JWT_ACCESS_SECRET , JWT_REFRESH_SECRET	Session tokens	long random strings

- Keys live only in the hosting provider's secret store / environment, never in the repository or the database. Access is limited to the people who deploy production.
- **Keep an offline backup of PII_ENCRYPTION_KEY** (e.g. a restricted password-manager vault). Losing it means losing every encrypted field. Atlas backups cannot recover them without the key.
- Without PII_ENCRYPTION_KEY the server falls back to a development key derived from the JWT secret, logs an error in production, and the admin Security page shows the key as not configured. **That state must never reach production.**

5.2 Rotation procedure

1. Generate a new key.
2. Set PII_ENCRYPTION_KEY = new key and PII_ENCRYPTION_KEY_PREVIOUS = old key. Restart all API instances. Values written with the old key keep decrypting. The start-up migration re-encrypts the fields in §4.3 with the new key.
3. In Admin → Platform Controls → Integrations, re-save any secrets that were entered there, so they are written with the new key.
4. Check Admin → Platform Controls → **Security & encryption**. Coverage should show no plaintext.
5. Remove PII_ENCRYPTION_KEY_PREVIOUS and restart.

Rotate at least yearly, and immediately if a key may have been exposed. The rotation path is covered by an automated test (`npm run test:field-encryption`).

6. Documents and files

6.1 Invoices and receipts

- Invoice/receipt PDFs are generated on request and never stored publicly.
- In the app, the PDF is served only to the authenticated customer who owns the subscription.
- Email attachments are fetched through a **signed link**: HMAC-SHA256 over subscription, user and expiry, compared in constant time, **valid for 7 days**. It needs API_PUBLIC_URL (https) and INVOICE_LINK_SECRET.

6.2 Deviation — driver identity documents (open item)

The spec requires driving licence, national ID and vehicle registration **images** to be stored with restricted access and served only through server-generated links that expire after 1 hour.

Current state: these images are uploaded to Cloudinary as public (unguessable, but permanent) URLs. Their **numbers** are encrypted (§4.3).

Planned fix: upload identity documents as Cloudinary `authenticated` assets. Serve them only through an admin/driver-authenticated API endpoint that returns a signed URL valid for 1 hour, and migrate the existing files. This changes the driver sign-up upload, the driver "My documents" screen and the admin document-review screen. It needs testing against the production Cloudinary account, which was not available in the development environment.

6.3 Other uploads

Meal photos, kitchen photos and Sanepid PDFs are business documents and stay on public Cloudinary URLs. **Recommendation:** require a signed-in user on the generic upload endpoints (`/api/v1/uploads/*`), and rate-limit them. Today they accept files without a login.

7. Access controls

- **Authentication.** Separate JWT access tokens per role (customer, vendor, driver, admin, office), with server-side refresh tokens that are revoked on logout. Customers, vendors and drivers sign in with phone OTP. Admins use email + password.
 - **Admin authorisation.** Every admin endpoint checks a module permission (view/create/edit/delete). The new platform-control endpoints also check the admin role (Super Admin, City Manager, Marketing, Finance...), as the Admin Controls Matrix specifies. Every control change is written to the **audit log** with before/after values and the admin's identity.
 - **Information visibility (ACM-131 → ACM-145).** Enforced on the server. A hidden field (customer phone for vendors, order value for drivers, etc.) is removed or masked in the API response itself, so it cannot be read from the app or the network. EU-mandated items (final price, B2B VAT invoice breakdown, B2C receipt total, GDPR notice) are not controls and cannot be hidden.
 - **Drivers** see only the deliveries on their own route. Only the driver who collected a box (collection PIN) can mark it delivered or failed.
 - **Vendors** see only their own orders, subscribers and settlements.
 - **Customers** can see and change only their own addresses, subscriptions, invoices and ratings.
 - **Consent and legal.** Marketing consent is opt-in and timestamped, and only opted-in emails are synced to Mailchimp. Every acceptance of a legal document (terms, privacy, cook agreement) is recorded with document version, language and time. A new version forces re-acceptance before the app can be used.
 - **Rate limiting** on the API and the OTP endpoints.
-

8. Pre-launch checklist (production)

#	Item	Owner
1	Load balancer / proxy: TLS 1.3 only, HTTP listener redirects or forwards with X-Forwarded-Proto	Hosting
2	NODE_ENV=production (turns on FORCE_HTTPS and HSTS)	Hosting
3	PII_ENCRYPTION_KEY set from a fresh random 32-byte key; offline backup stored	Hosting + DailyMealBox
4	INVOICE_LINK_SECRET set; API_PUBLIC_URL set to the https:// API URL	Hosting
5	MongoDB Atlas: encryption at rest confirmed (MongoDB-managed or AWS KMS) in writing ; IP access list; least-privilege DB user; mongodb+srv:// connection	Hosting + DailyMealBox
6	Android wrapper: cleartext traffic disabled	Mobile
7	Admin → Platform Controls → Security & encryption shows all green, with no plaintext in coverage	Admin
8	Decision on phone/email encryption (§4.4) and schedule for driver-document access (§6.2)	DailyMealBox

9. Verification

- **Live check:** Admin → Platform Controls → **Security & encryption**. It reports HTTPS redirect, HSTS, the API URL scheme, WSS, database TLS/Atlas, key configuration, and, for every field in §4.3, a count of encrypted vs plaintext values. It shows no values and no key material.
- **Automated tests:** `npm run test:field-encryption` covers every read and write path, the migration, key rotation and driver ID numbers. `npm run test:failed-delivery` covers driver ownership of deliveries.